Zürcher Hochschule
für Angewandte Wissenschaften

School of
Engineering
InES Institute of
Embedded Systems

# Embedded Software Engineering

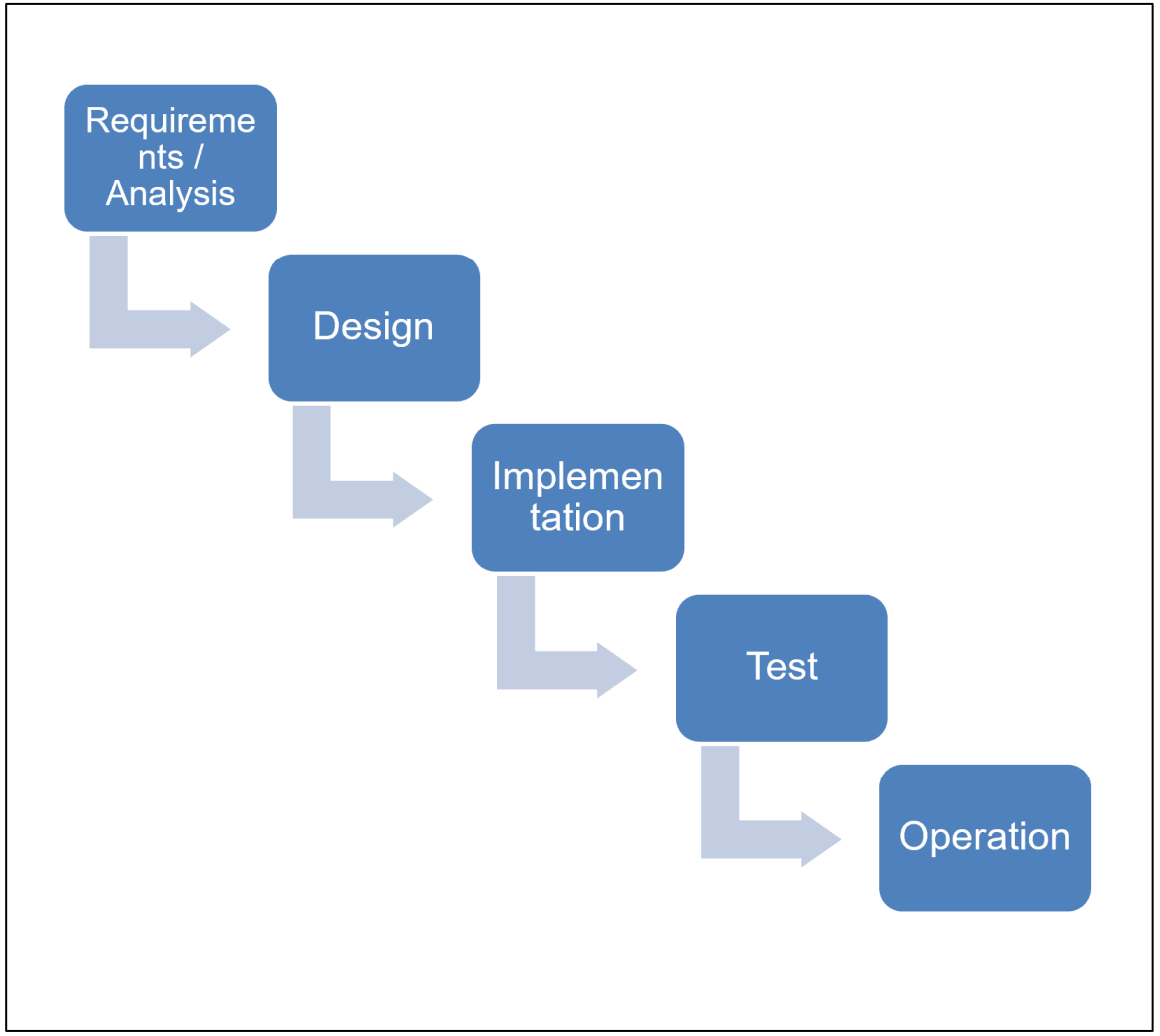
**Simon Künzli, Tobias Welti**

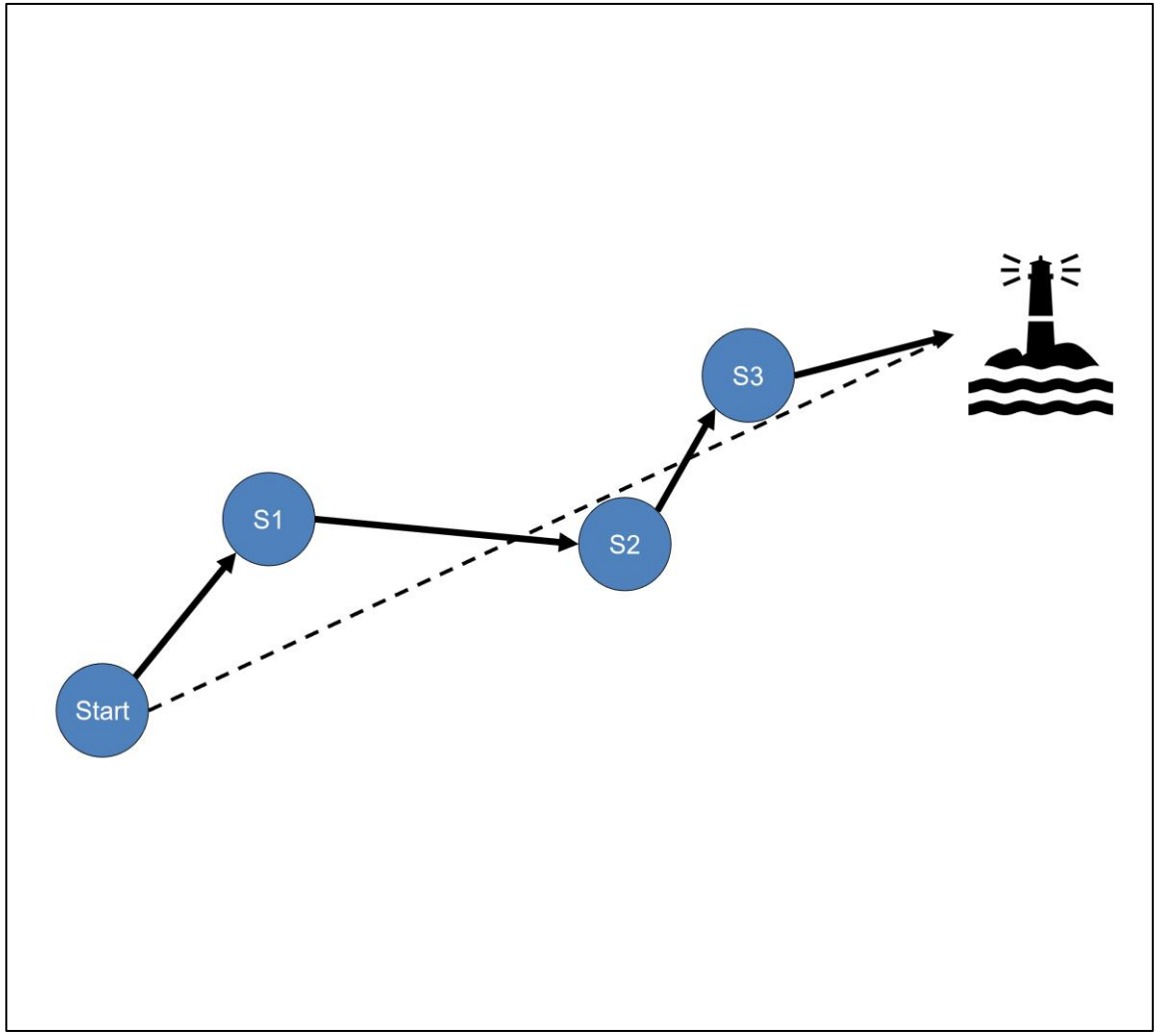
**HS 2025**

# Recap: Abstraction Levels

| Level | Hardware | Software |
|---|---|---|
| System | Components and Sub-systems | Tasks that describe the system functionality |
| Module | Processors, MCUs, ASICs, Buses, Peripherals, Memory, Storage | Interacting SW modules |
| Block | Counter, Registers, ALUs | Program sequences, loops |
| Logic | Logic gates, Flip-Flops | Instructions (e.g. ADDS R1, R2, R3) |
| Device | Electronic components, transistors | Machine Code (e.g. 0x21ff) |

# Recap: Development Models



# Agenda HS25

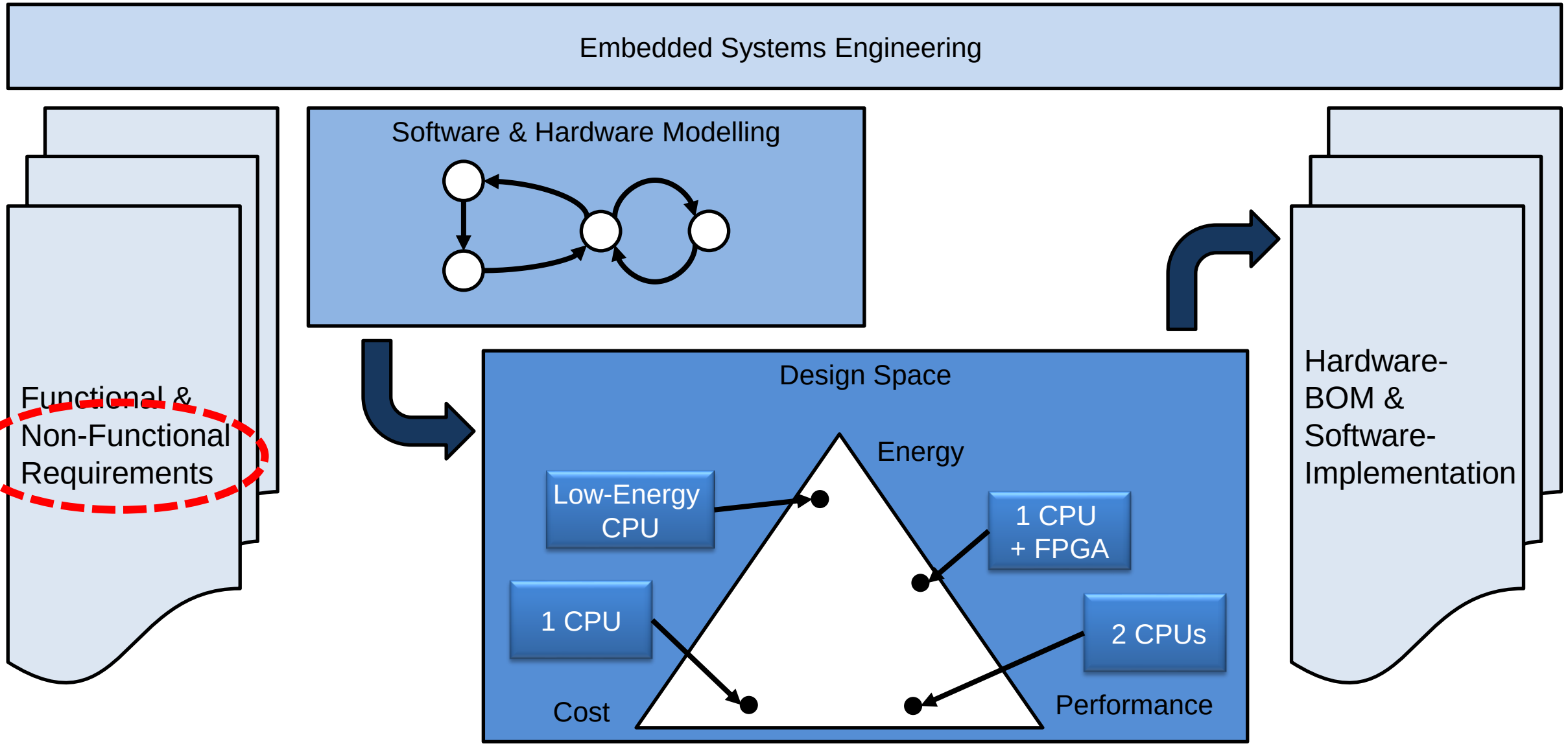
Zürcher Hochschule für Angewandte Wissenschaften
School of Engineering
InES Institute of Embedded Systems

Vorlesung (welo)
Vorlesung (kuex)
Übungen / Praktika

Stand: 27.08.25

| wk # | Do, 10:00 - 11:35 | Do, 08:00 – 09:35 / Do 12:00 – 13:35 | |
|---|---|---|---|
| 01 15.09. | Modulorganisation & Einführung Embedded Systems | No Lab | |
| 02 22.09. | SW-Paradigmen für Embedded Systems | No Lab | P1: First steps U96 |
| 03 29.09. | Requirements für Embedded Systems | P1: First steps U96 | P1: Finish Lab |
| 04 06.10. | Software-Entwicklung für ES (Modellierung, SW-Specs) | P1: Finish Lab | U1: Requirements |
| 05 13.10. | Entwicklungsprozesse / CI/CD | U1: Requirements | U2: Modeling |
| 06 20.10. | Non-functional Requirements | U2: Modeling | U3: Implementation & Test |
| 07 27.10. | Visite Fertigung Siemens | Visite Fertigung Siemens | |
| 08 03.11. | Fokus "Energie« / andere Prozessoren? | U3: Implementation & Test | P2: Energy Consumption |
| 09 10.11. | Fokus "Performance" | P2: Energy Consumption | P3: AES in SW, incl. Measurement on R5 |
| 10 17.11. | Einführung FPGA | P3: AES in SW, incl. Measurement on R5 | P3: Finish Lab |
| 11 24.11. | Einführung Design Space Exploration (DSE) | P3: Finish Lab | P4: AES in FPGA and HW-Accelerator |
| 12 01.12. | DSE: Kostenfunktionen und Suche im Entwurfsraum | P4: AES in FPGA and HW-Accelerator | P4: Finish Lab |
| 13 08.12. | Einführung RTOS | P4: Finish Lab | P5: DSE |
| 14 15.12. | Dual-Prozessor-Systeme mit Uniform Memory Access | P5: DSE | No Lab |

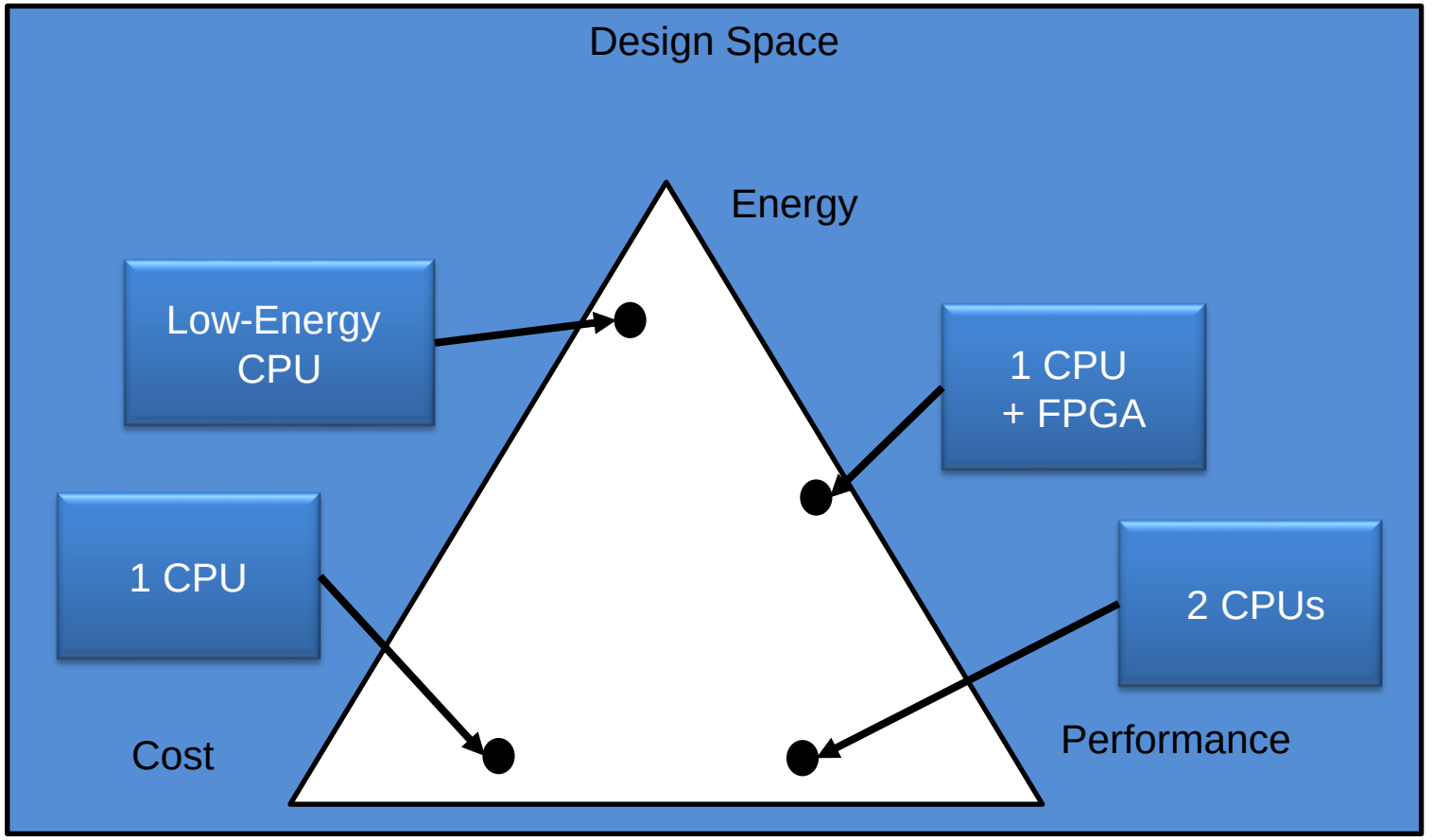
# Advance Organizer ESE



# Motivation

- **Imagine an app that has every feature you asked for,
but it's slow, crashes often, and looks terrible… is it successful?»**

# Motivation


Design Space
Energy
Low-Energy CPU
1 CPU + FPGA
1 CPU
2 CPUs
Cost
Performance

# Goals

**After today's lesson, you should be able to …**

- **… describe the difference between functional and non-functional requirements**
- **… explain different categories and aspects of non-functional requirements**
- **… write non-functional requirements**
- **… distinguish between a functional and a non-functional requirement**
- **… identify and categorize non-functional requirements**

# Definition «Non-Functional Requirement»

- What's the difference between software that works and software that works *well*? Non-functional requirements.
- Non-functional requirements describe what the system *should be* instead of what the system should *do*.
- **A Definition attempt:** A non-functional requirement is a requirement for a product that specifies criteria that can be used to judge a system but does NOT describe the desired behavior (or functionality) of the system.
- In literature, NFRs are also called "Quality attributes", "Constraints (limitations)", or "technical requirements", "-ilities"
- As with other requirements, NFRs must follow the same rules (see next slide).

# Recap Week 3: Requirements – Characteristics

- A requirement needs to meet several criteria to be considered a "good requirement" (see also [1]).

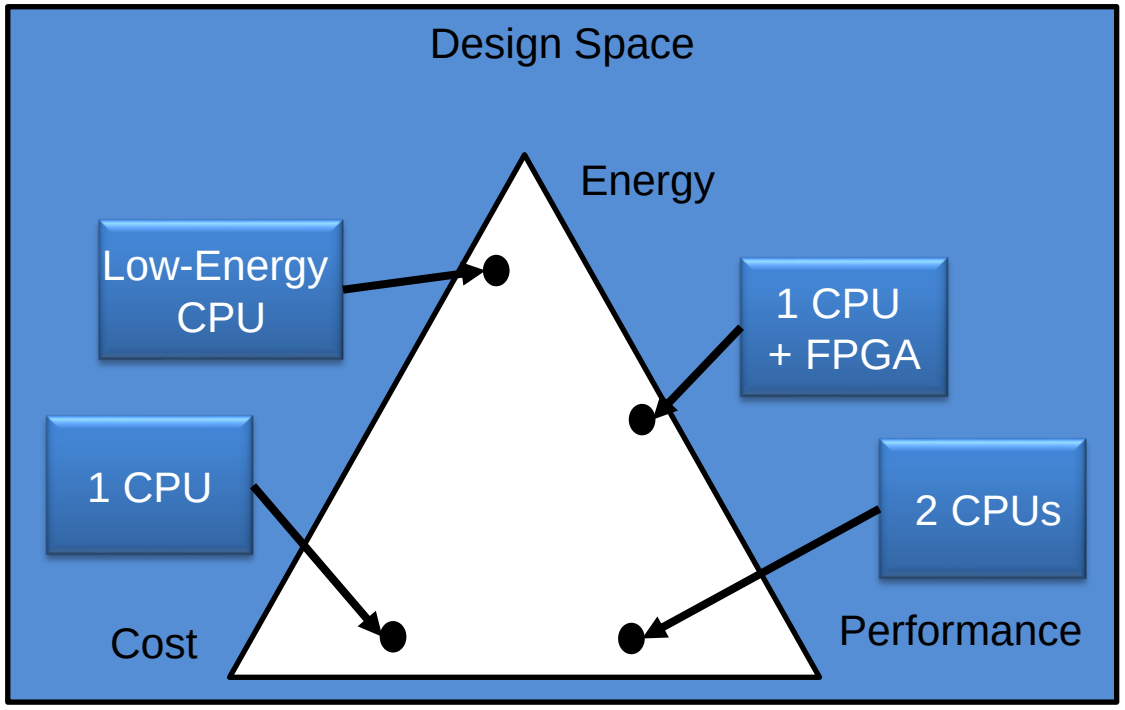
- Good requirements should, e.g., have the following characteristics:

- Unambiguous
- Testable (verifiable)
- Clear (concise, terse, simple, precise)
- Correct
- Understandable
- Feasible (realistic, possible)
- Independent
- Atomic
- Necessary
- Implementation-free (abstract)

In addition, we usually assign priorities to requirements, e.g., 0 – 3.

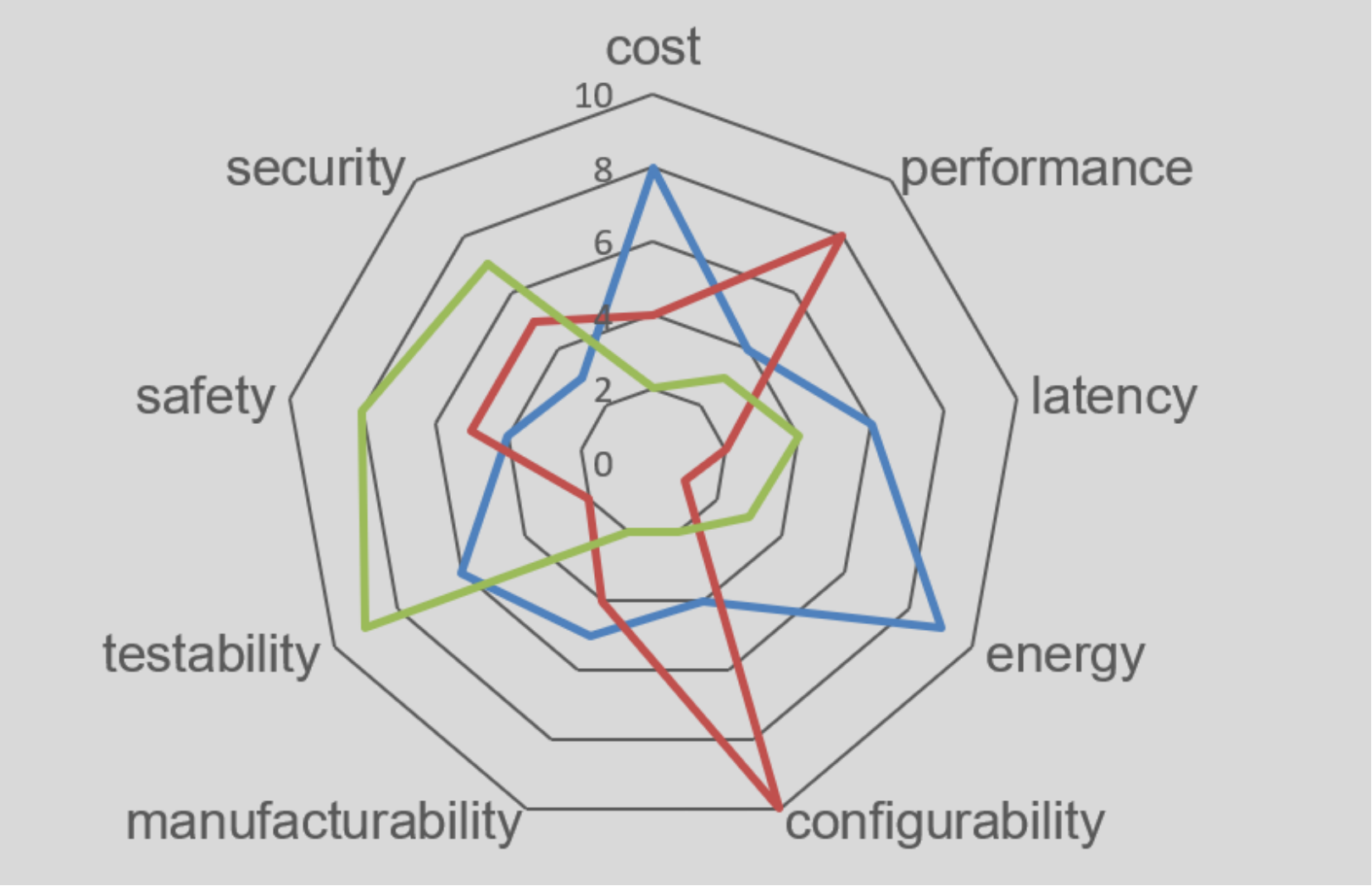
[1] Hull, Elizabeth, Kenneth Jackson, and Jeremy Dick. Requirements Engineering, London: Springer, 2005.

# Most popular NFRs include …




- **Energy → see Lecture in 2 Weeks**
- **Performance → see Lecture in 3 Weeks**
  - response time, transaction rates, throughput, etc.
- **(Product) Cost → see Math class ☺**
  - Product Cost =
    Bill-of-Material (BOM)
    + Software License Fees
    + Manufacturing Cost
    + Manufacturing Overhead

# There are more NFR types …



# Are there Categories of Requirements for ES?

**A first attempt:**

- **Function** → **Functional Requirements**
- **Efficiency**
- **Dependability**
- **Security**

} → **Non-Functional Requirements**

# Efficiency, Dependability & Security

## Efficiency

Energy

Cost

Development Time

Performance

Memory

Weight

Space

## Dependability

Reliability

Availability

Integrity

Standard compliance

Safety

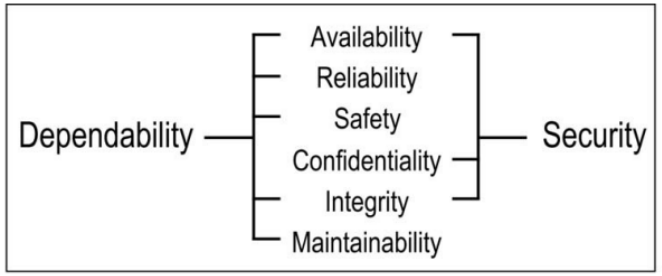
Maintainability

## Security

Confidentiality

Availability

Integrity



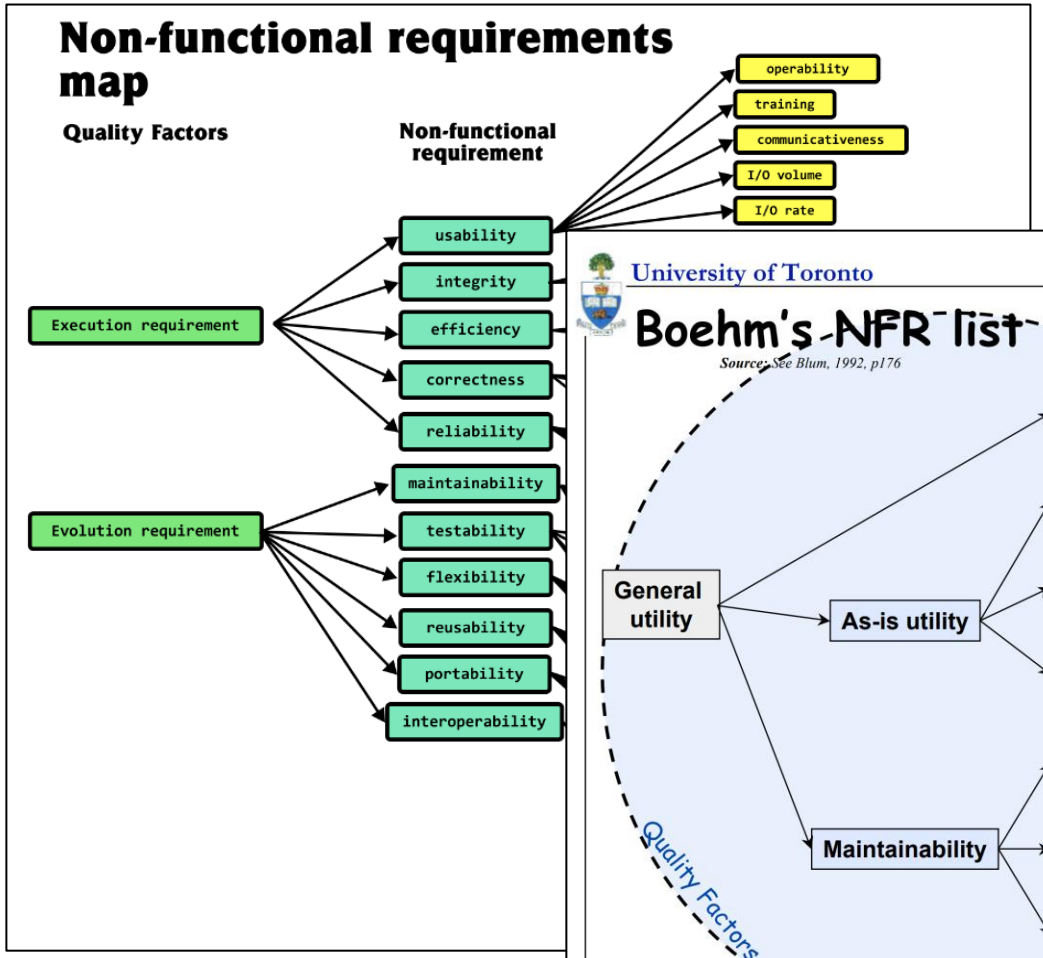

Source: A. Avizienis, J. . -C. Laprie, B. Randell and C. Landwehr, "Basic concepts and taxonomy of dependable and secure computing," in IEEE Transactions on Dependable and Secure Computing, vol. 1, no. 1, pp. 11-33, Jan.-March 2004, doi: 10.1109/TDSC.2004.2.

# ISO/IEC 25010

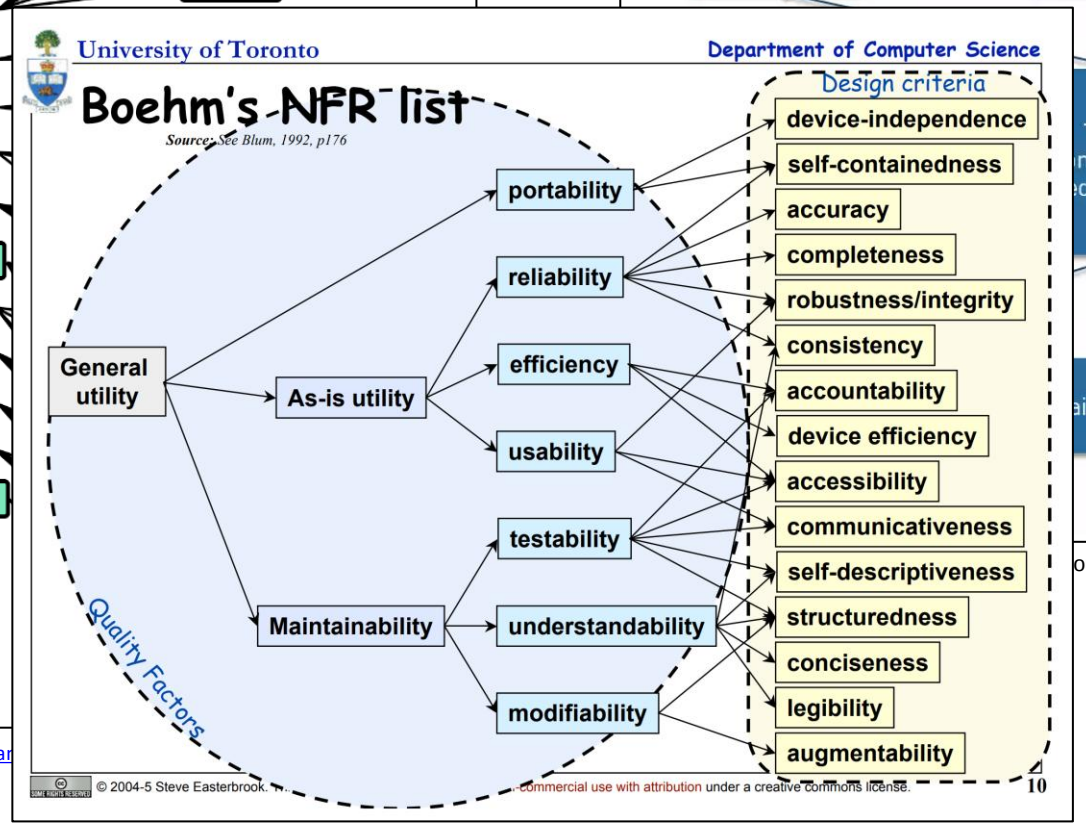
| SOFTWARE PRODUCT QUALITY | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| **FUNCTIONAL SUITABILITY** | **PERFORMANCE EFFICIENCY** | **COMPATIBILITY** | **INTERACTION CAPABILITY** | **RELIABILITY** | **SECURITY** | **MAINTAINABILITY** | **FLEXIBILITY** | **SAFETY** |
| FUNCTIONAL COMPLETENESS<br>FUNCTIONAL CORRECTNESS<br>FUNCTIONAL APPROPRIATENESS<br>iso25000.com | TIME BEHAVIOUR<br>RESOURCE UTILIZATION<br>CAPACITY | CO-EXISTENCE<br>INTEROPERABILITY | APPROPRIATENESS RECOGNIZABILITY<br>LEARNABILITY<br>OPERABILITY<br>USER ERROR PROTECTION<br>USER ENGAGEMENT<br>INCLUSIVITY<br>USER ASSISTANCE<br>SELF-DESCRIPTIVENESS | FAULTLESSNESS<br>AVAILABILITY<br>FAULT TOLERANCE<br>RECOVERABILITY | CONFIDENTIALITY<br>INTEGRITY<br>NON-REPUDIATION<br>ACCOUNTABILITY<br>AUTHENTICITY<br>RESISTANCE | MODULARITY<br>REUSABILITY<br>ANALYSABILITY<br>MODIFIABILITY<br>TESTABILITY | ADAPTABILITY<br>SCALABILITY<br>INSTALLABILITY<br>REPLACEABILITY | OPERATIONAL CONSTRAINT<br>RISK IDENTIFICATION<br>FAIL SAFE<br>HAZARD WARNING<br>SAFE INTEGRATION |

Source: https://iso25000.com/index.php/en/iso-25000-standards/iso-25010

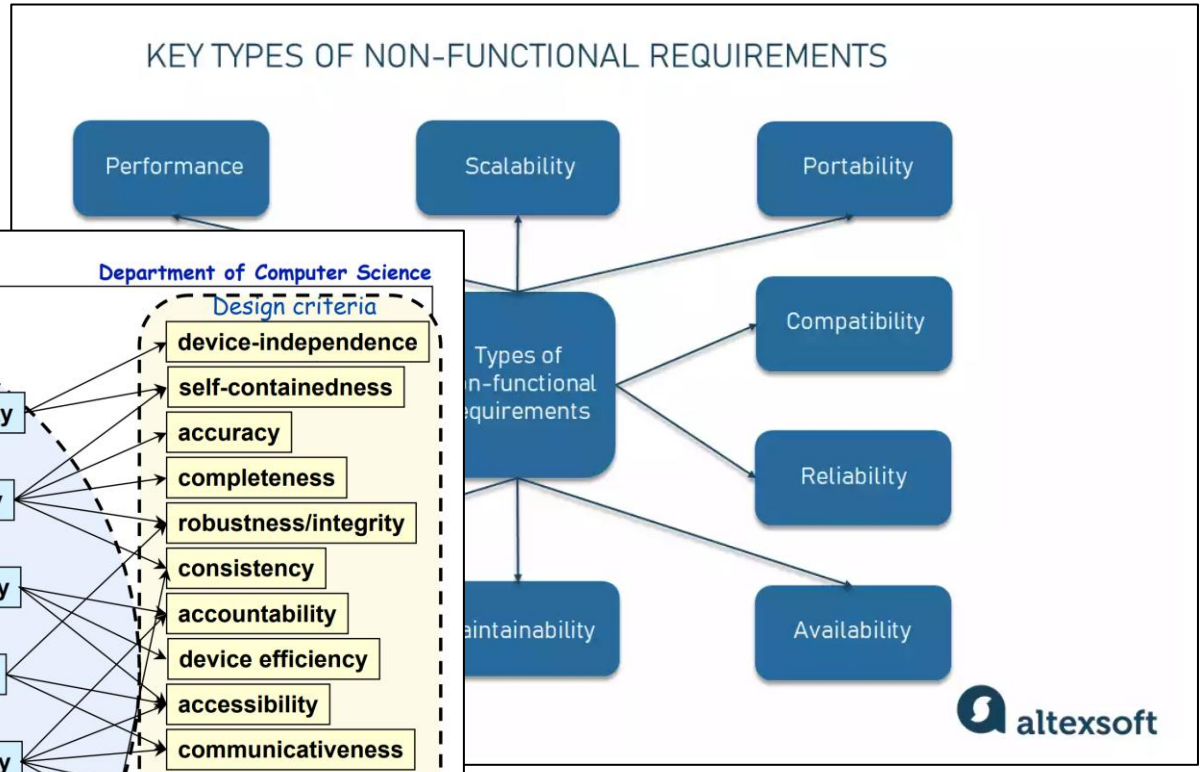
# Other approaches for further classification or categorization




Source: https://khalilstemmler.com/articles/object-oriented/ar




Source: https://cs.toronto.edu/~sme/CSC340F/slides/16-NFRs.pdf




Source: https://www.altexsoft.com/blog/non-functional-requirements/

# Classification of NFRs on Multiple Dimensions

- **Development Classification**
  - **Qualities** define the properties and characteristics that the delivered system should demonstrate
  - **Constraints** are the limitations, standards, and environmental factors that must be taken into account in the solution

- **Temporal Classification**
  - **Execution** Requirements (observable during operation)
  - **Evolution** Requirements (observable during the development)

- **Measurable Classification**
  - **Quantifiable** Requirements
  - **Qualifiable** Requirements

# Mapping NFRs to Categories

| Requirement | Qualifiable | Quantifiable |
|---|---|---|
| **Execution** | Standard Compliance | Current consumption<br>Security<br>Performance<br>Usability |
| **Evolution** | SW Licenses<br>Process Compliance | Development time<br>Project Cost<br>Testability<br>Scalability |

# Execution NFRs

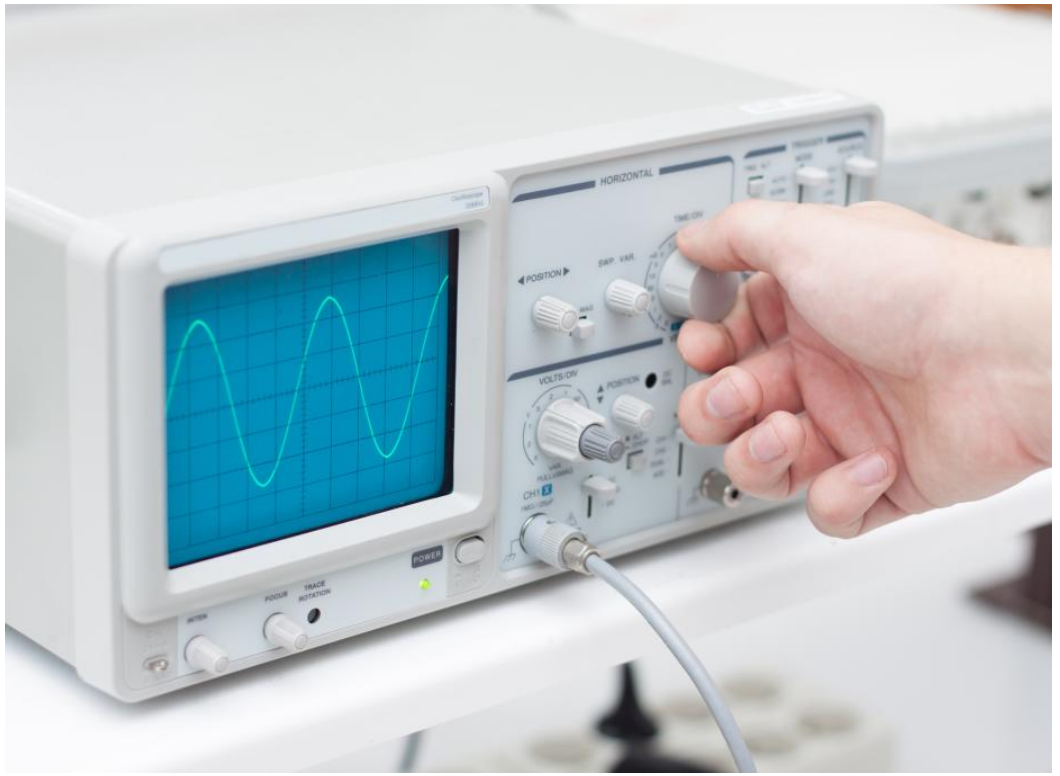
**Execution NFRs are relevant for the Embedded System in *operation*.**

- **Examples include:**
  - Energy requirements
  - **Physical requirements** (size, weight)
  - Performance requirements (response time, execution time)
  - Memory requirements (RAM, ROM/Flash)
  - Product Cost requirements
  - Security requirements
  - Complexity requirements

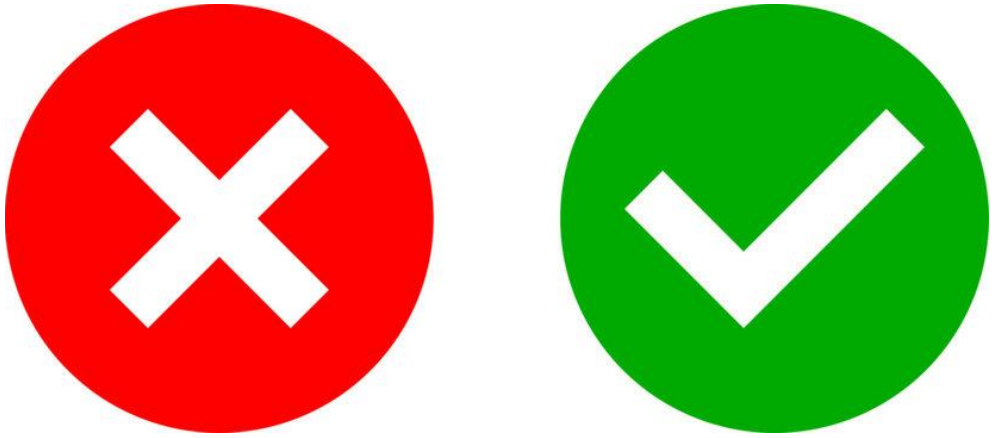
# Evolution NFRs

**Evolution NFRs are relevant for the Embedded System in the *development phase*.**

- **Examples include:**
  - Manufacturability requirements
  - Extensibility requirements
  - Usability requirements
  - Modifiability requirements
  - Portability requirements
  - **Testability requirements**
  - Configurability requirements
  - Scalability requirements
  - Development Time requirements



# Qualifiable NFRs



- Regulatory — Do local, regional, provincial, or state laws or regulations dictate some aspect of what we're legally allowed to do?
- Localization — Will the system adapt to measurement systems, time zones, and other locally-specific concepts in other states or provinces?
- Legal – There may be legal issues involving privacy of information, intellectual property rights, export of restricted technologies, etc.

# Quantifiable NFRs

- Capacity — What size can our program code occupy? How much data can we store? Will we ever run out of storage space?
- Performance — How fast should requests process? How many requests can we handle at any given time? What should we expect for response times when the system is under heavy load?
- Usability – Requirements about how difficult it will be to learn and operate the system. The requirements are often expressed in learning time or similar metrics.
- Security – Requirements about the protection of your system and its data. The measurement can be e.g., the required effort, skill level, or time, to break into the system.
- Reliability – Requirements about how often the software fails. The measurement is often expressed in MTBF (mean time between failures) and can be calculated based on the design data. How long does it take for a system to come back online after a failure?

# Constraints

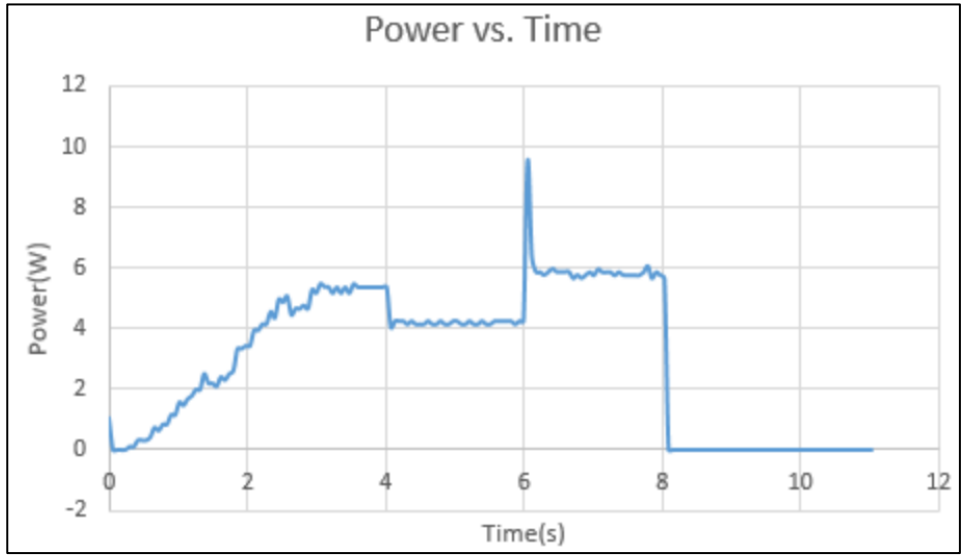
**Constraints are the limitations, standards, and environmental factors that must be fulfilled when developing an Embedded System.**


- **Examples include:**
  - Standard compliance
  - Company development processes
  - Programming Language constraints
  - Platform reuse constraints
  - Intellectual Property constraints

# Deriving Measurable Criteria

**For some NFRs, deriving measurable criteria is straight-forward**

- **Energy consumption may be measured using a power analyzer**

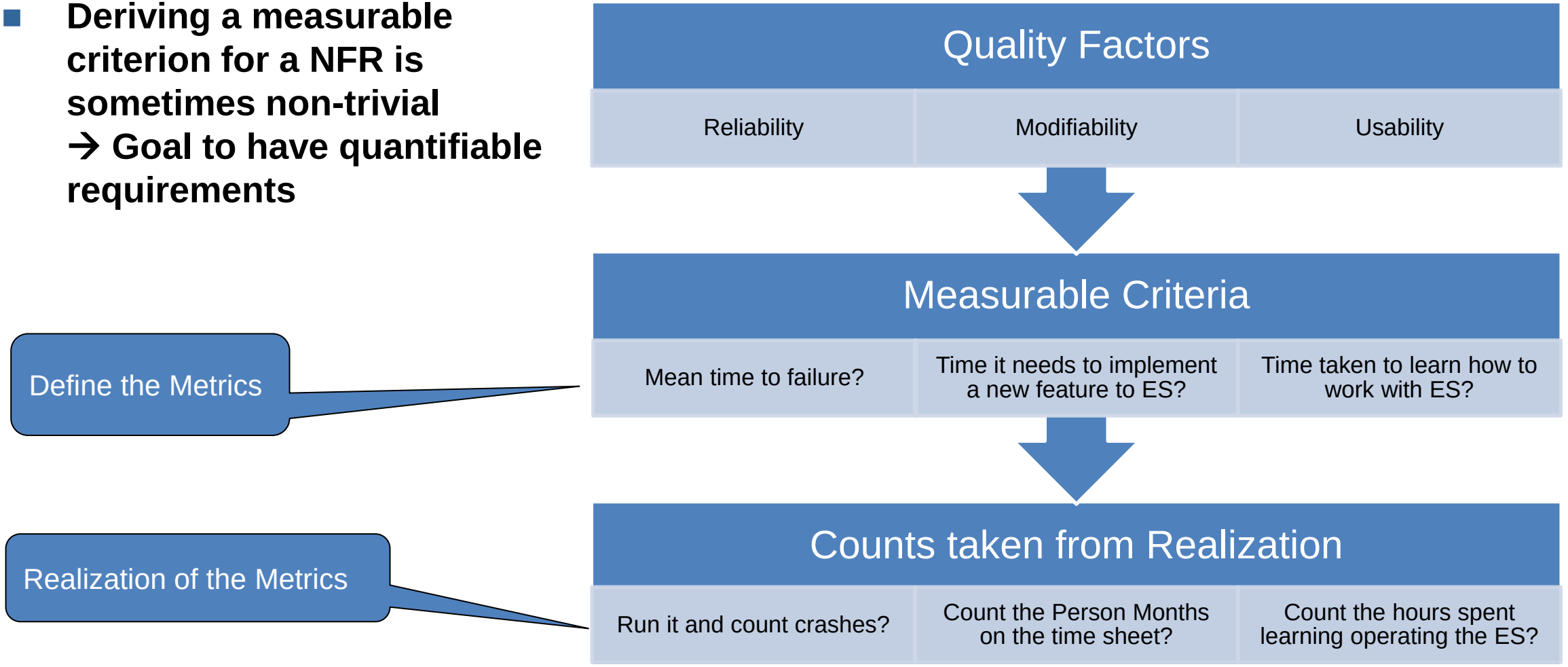



- **Memory consumption may be derived from *.lst/*.map file**

```
Memory Map of the image

  Image Entry point : 0x08000865

  Load Region LR_IROM1 (Base: 0x08000000, Size: 0x00003590, Max: 0x00200000, ABSOLUTE)

    Execution Region ER_IROM1 (Exec base: 0x08000000, Load base: 0x08000000, Size: 0x00003588, Max: 0x00200000, ABSOLUTE)

    Exec Addr    Load Addr    Size         Type   Attr      Idx    E Section Name        Object

    0x08000000   0x08000000   0x000001ac   Data   RO          50    RESET               startup_ctboard.o
    0x080001ac   0x080001ac   0x0000003c   Code   RO         119    !!!scatter          c_pu.l(__scatter.o)
    0x080001e8   0x080001e8   0x0000001a   Code   RO         284    !!handler_copy      c_pu.l(__scatter_copy.o)
    0x08000202   0x08000202   0x00000002   PAD
    0x08000204   0x08000204   0x0000001c   Code   RO         286    !!handler_zi        c_pu.l(__scatter_zi.o)
    0x08000220   0x08000220   0x00000002   Code   RO         170    .ARM.Collect$$libinit$$00000000  c_pu.l(libinit.o)
    0x08000222   0x08000222   0x00000000   Code   RO         180    .ARM.Collect$$libinit$$00000002  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         182    .ARM.Collect$$libinit$$00000004  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         184    .ARM.Collect$$libinit$$00000006  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         187    .ARM.Collect$$libinit$$0000000C  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         189    .ARM.Collect$$libinit$$0000000E  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         191    .ARM.Collect$$libinit$$00000010  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         194    .ARM.Collect$$libinit$$00000013  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         196    .ARM.Collect$$libinit$$00000015  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         198    .ARM.Collect$$libinit$$00000017  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         200    .ARM.Collect$$libinit$$00000019  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         202    .ARM.Collect$$libinit$$0000001B  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         204    .ARM.Collect$$libinit$$0000001D  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         206    .ARM.Collect$$libinit$$0000001F  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         208    .ARM.Collect$$libinit$$00000021  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         210    .ARM.Collect$$libinit$$00000023  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         212    .ARM.Collect$$libinit$$00000025  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         214    .ARM.Collect$$libinit$$00000027  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         218    .ARM.Collect$$libinit$$0000002E  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         220    .ARM.Collect$$libinit$$00000030  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         222    .ARM.Collect$$libinit$$00000032  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000000   Code   RO         224    .ARM.Collect$$libinit$$00000034  c_pu.l(libinit2.o)
    0x08000222   0x08000222   0x00000002   Code   RO         225    .ARM.Collect$$libinit$$00000035  c_pu.l(libinit2.o)
    0x08000224   0x08000224   0x00000002   Code   RO         259    .ARM.Collect$$libshutdown$$00000000  c_pu.l(libshutdown.
```

# Deriving Measurable Criteria

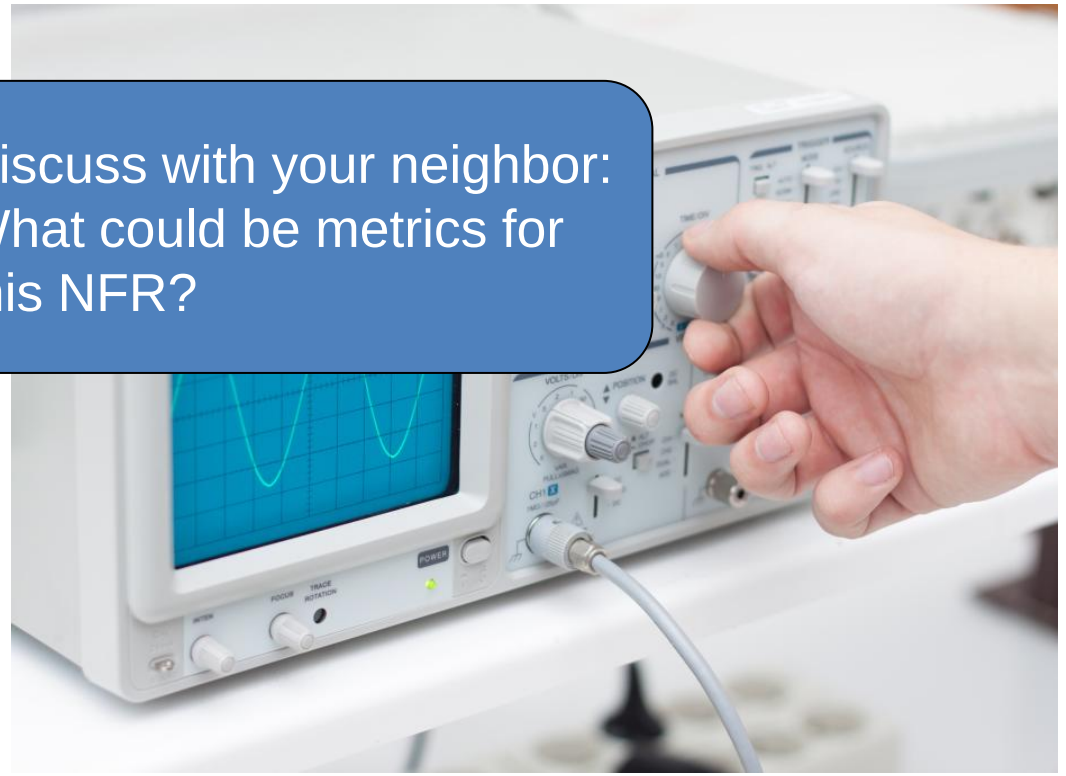
- **Deriving a measurable criterion for a NFR is sometimes non-trivial**
  **→ Goal to have quantifiable requirements**



# Evolution NFRs

**Evolution NFRs are relevant for the Embedded System in the *development phase*.**

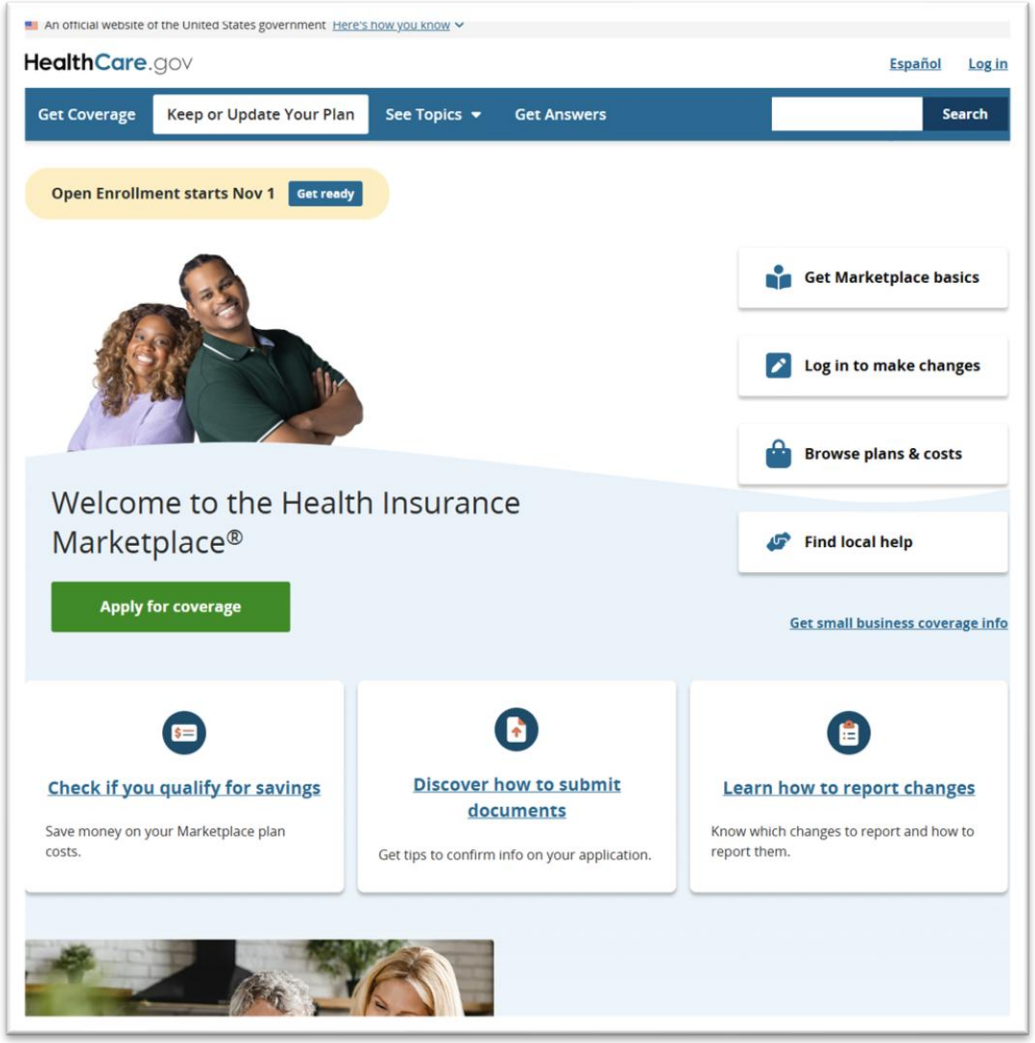
- **Examples include:**
  - **Manufacturability requirements**
  - Extensibility requirements
  - Usability requirements
  - Modifiability requirements
  - Portability requirements
  - Testability requirements
  - Configurability requirements
  - Scalability requirements
  - Development Time requirements



# Why are NFRs important?

| NFR | Why Does It Matter in Embedded Systems? |
|---|---|
| **Timing (Real-Time Constraints)** | Missed deadlines can cause physical harm or failure. |
| **Safety** | Often safety-critical environments (aerospace, medical, automotive). |
| **Reliability / Robustness** | Must operate continuously in unpredictable conditions. |
| **Fault Tolerance** | Must continue operating under partial hardware/software failure. |
| **Maintainability** | Embedded code often has long lifespans, Embedded Systems have long product lifetime. |
| **Testability** | Hardware–software integration complicates testing of edge cases. |

# Case Study – Unmet NFRs




Source: https://healthcare.gov

- **When healthcare.gov was launched in 2013, it crashed almost immediately and remained unusable for weeks**

- **Users faced issues such as:**
  - Long page load times (sometimes over 8 minutes)
  - Site crashes during registration or checkout
  - Error messages without explanations
  - Frozen pages and data loss
  - Only ~6 people were able to successfully register on launch day out of millions of visitors!

# Case Study – Violated/Unmet NFRs

| NFR Category | Issue | Example of Violation |
|---|---|---|
| **Performance** | The system was not stress-tested for expected traffic levels. | The site could only handle ~1,000 concurrent users, though over 250,000 tried to access it at launch. |
| **Scalability** | No load balancing or horizontal scaling mechanisms were properly configured. | Infrastructure couldn't dynamically scale under load. |
| **Reliability** | System crashed repeatedly; sessions timed out or data was lost mid-registration. | Frequent "500" errors and incomplete transactions. |
| **Usability** | Complex, multi-step registration process; unclear error messages. | Users didn't understand what failed or how to retry. |
| **Maintainability** | Codebase was fragmented across multiple contractors, poorly integrated. | Small changes broke other modules. |
| **Security** | Concerns about personal data handling and incomplete vulnerability testing. | Security reviews were rushed before launch. |

# Challenges with NFRs

- **They can be difficult to write such that they can be verified.**
- **They can be difficult/time-consuming to understand and implement.**
- **They can be time-consuming and expensive to test.**
- **They can impact the functionality of the system if not properly implemented.**
- **They can be contradictory, making it necessary to make decisions on possible trade-offs.**

# Trade-offs

- **Discuss *trade-offs* between NFRs with your neighbor:**

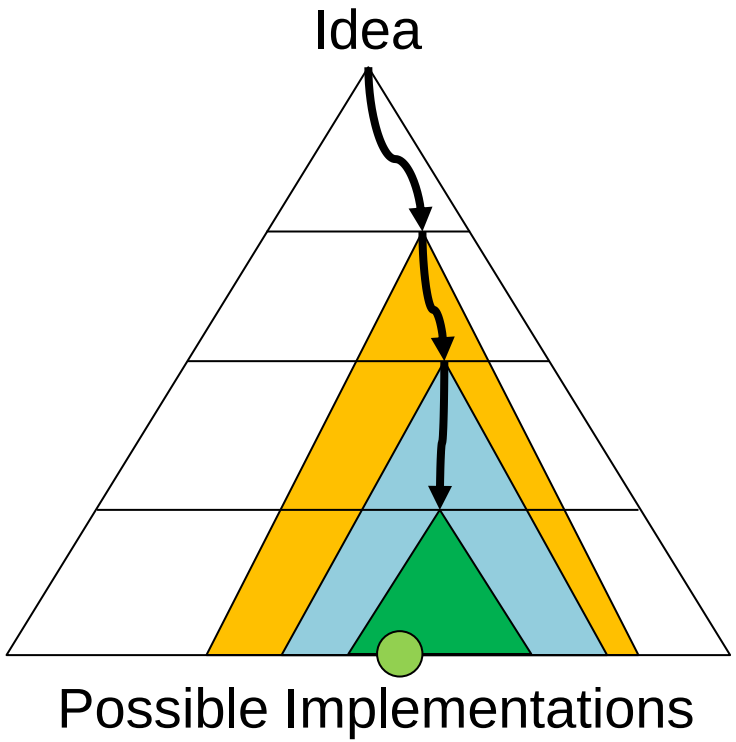
- **Examples trade-offs:**
  - Security vs. Usability
  - Performance vs. Maintainability
  - Performance vs. Security
  - Maintainability vs. Time-to-Market
  - Reliability vs. Cost
  - Safety vs. Flexibility
  - Portability vs. Performance

- **Find examples, where you would judge the trade-offs differently**
  - E.g., how would trade-offs differ for a medical device vs. a mobile game?

# Examples with Traffic Light Control System

- **R1: «The product cost of a single traffic light shall not be higher than 3000 CHF @1000 pieces.»**
- **R2: «The traffic light control system shall fit a standard traffic control housing according to the SN XXXX, V1.x, 2023»**
- **R3: «The traffic light control shall show a green light to only one lane at a time on the crossroad.»**
- **R4: «The traffic light control system shall be extensible to enable the use with a 5-road intersection.»**
- **R5: «It shall be possible to manufacture the traffic light in a fully automated process.»**
- **R6: «The MTBF for a single traffic light shall be larger than 500 years.»**

# The Role of NFRs on Embedded System Design




- **All possible implementations fulfill the functional requirements**
- **NFRs are responsible for guiding the design towards the optimal solution**
- **Constraints NFRs represent guardrails to exclude solutions that cannot meet them**
- **The earlier in the design process the compliance to an NFR can be confirmed, the better**
- **Comparison between two solutions concerning compliance to an NFR might be useful**

# Summary

- **Non-functional requirements describe what the system should be instead of what the system should do**
- **There exists a wide area of non-functional requirements (NFRs) and many classification approaches**
- **NFRs ≠ “nice to have”**
- **NFRs can be categorized into qualities and constraints**
- **NFRs can be categorized into quantifiable and qualifiable requirements**
- **NFRs can be categorized into execution and evolution requirements**
- **It’s important to have tools available to quantify compliance to NFRs whenever possible**